FitnessProgressor

Service Layer Design

Samed Ganibegovic

Professor Gradecki

SWDV 691: SOFTWARE DEVELOPMENT CAPSTONE

3/22/2026

1. Overview

The backend service for FitnessProgressor will be implemented as a RESTful API using *Node.js* and *Express*. This service layer acts as the communication bridge between the frontend interface and the *PostgreSQL* database. All data retrieval, updates, and analytical operations will be handled through this API. The goal is to maintain a clean separation between the user interface and the underlying business logic so that the application remains organized, scalable, and easy to maintain.

The backend is divided into three layers. The *Route Layer* defines the available API paths and the HTTP methods associated with them. The *Controller Layer* receives incoming requests, validates the data, and returns appropriate responses. The *Service Layer* contains the core logic for interacting with the database and performing analytics such as trend detection, plateau analysis, and forecasting. This structure ensures that each part of the backend has a clear responsibility and that the system remains easy to extend as the project grows.

The FitnessProgressor backend service layer is implemented as a RESTful API that mediates all communication between the user interface and the *PostgreSQL* database. The service layer ensures:

- A clean separation between UI and data
- Consistent access patterns
- Centralized business logic for analytics
- Scalable and maintainable architecture

1. Route Layer

Defines the URL paths and HTTP methods available to the client.

2. Controller Layer

Handles incoming requests, validates input, and returns responses.

3. Service Layer

Contains the core business logic, interacts with the database, and performs analytics such as trend detection, plateau analysis, and forecasting.

All endpoints return JSON and follow RESTful conventions.

2. Endpoint Specifications

The following sections describe all of the routes and endpoints that make up the FitnessProgressor service layer. Each endpoint includes its purpose, how it supports the application's user flows, and example request and response formats. These examples illustrate how the frontend will communicate with the backend during normal operation.

Below is the complete list of endpoints required for the FitnessProgressor MVP.

Each endpoint includes:

- Method
- URL
- Purpose
- Persona connection
- Example request
- Example success response
- Example error response
- A simple diagram showing UI → API → Database

3. Authentication Route

1. Login User

Method: POST
URL: /auth/login

***Purpose*:**
Authenticate a user and return a session token.

Persona Connection:
All personas must log in before accessing their dashboard, logging workouts, or viewing insights.

Example Request:

```
json
{
  "email": "mia@example.com",
  "password": "password123"
}
```

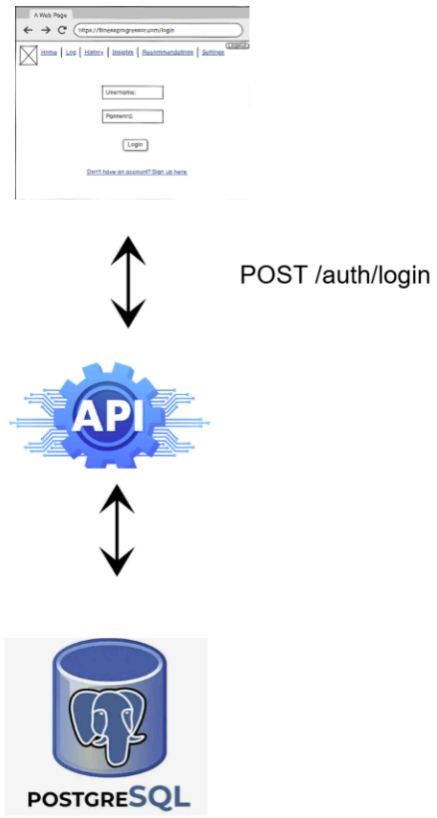
Success Response (200):

```
json
{
  "userId": 1,
  "name": "Motivated Mia",
  "token": "abc123.jwt.token"
}
```

Error Response (401):

```
json
{
  "error": "Invalid email or password."
}
```

Diagram:



4. User Routes (Settings Page)

2. Get User Profile

Method: GET
URL: /users/{userId}

Purpose:
Retrieve user profile information for the Settings page.

Persona Connection:

- Mia: checks her goal and units
- Daniel: aligns goals with performance
- Emily: quick review of settings

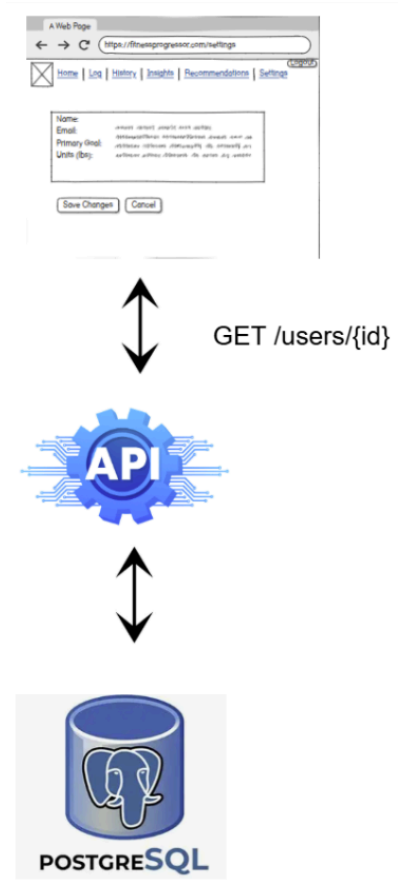
Success Response (200):

```
json
{
  "userId": 1,
  "name": "Motivated Mia",
  "email": "mia@example.com",
  "primaryGoal": "Strength",
  "units": "lbs"
}
```

Error Response (404):

```
json
{
  "error": "User not found."
}
```

***Diagram*:**



3. Update User Profile

Method: PUT
URL: /users/{userId}

Purpose:
Update user settings such as name, email, primary goal, and units.

Example Request:

```
json
{
  "name": "Motivated Mia",
  "email": "mia@example.com",
  "primaryGoal": "Hypertrophy",
  "units": "kg"
}
```

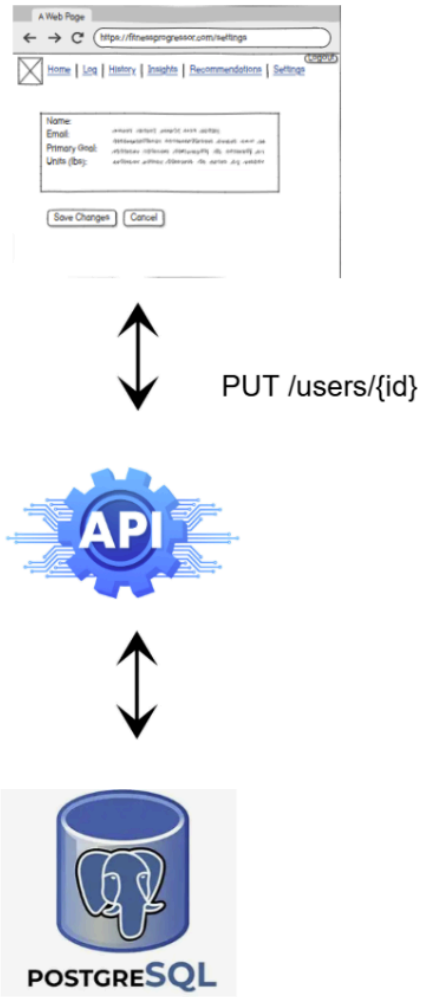
Success Response (200):

```
json
{
  "message": "User updated successfully."
}
```

Error Response (400):

```
json
{
  "error": "Invalid units. Allowed values: lbs, kg."
}
```

Diagram:



5. Workout Routes (Log Page + History Page)

4. Log a New Workout

Method: POST
URL: /workouts

Purpose:
Create a new workout and associated sets.
This powers the **Log** page and the **Success Modal**.

Persona Connection:

- Mia: logs today's workout
- Daniel: logs detailed sessions
- Emily: logs quickly before work

Example Request:

```
json
{
  "userId": 1,
  "workoutDate": "2026-03-15",
  "sets": [
    { "exerciseId": 1, "reps": 8, "weight": 65 },
    { "exerciseId": 1, "reps": 8, "weight": 65 }
  ]
}
```

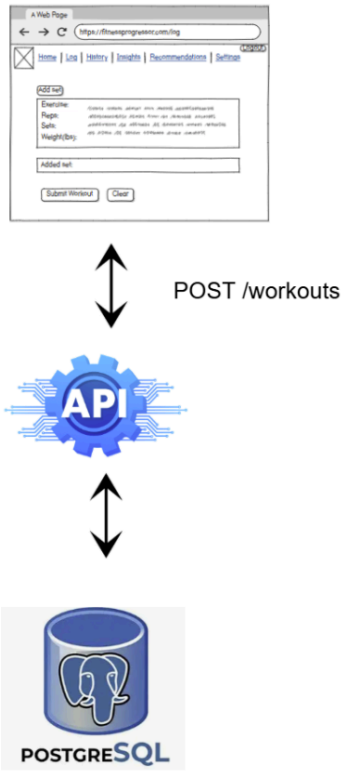
Success Response (201):

```
json
{
  "workoutId": 10,
  "message": "Workout logged successfully."
}
```

Error Response (400):

```
json
{
  "error": "At least one set is required."
}
```

***Diagram*:**



5. Get Workout History

Method: GET
URL: /workouts

Purpose:
Retrieve a user's workout history for the **History** page.

Example Request:

GET /workouts?userId=1&from=2026-03-01&to=2026-03-15

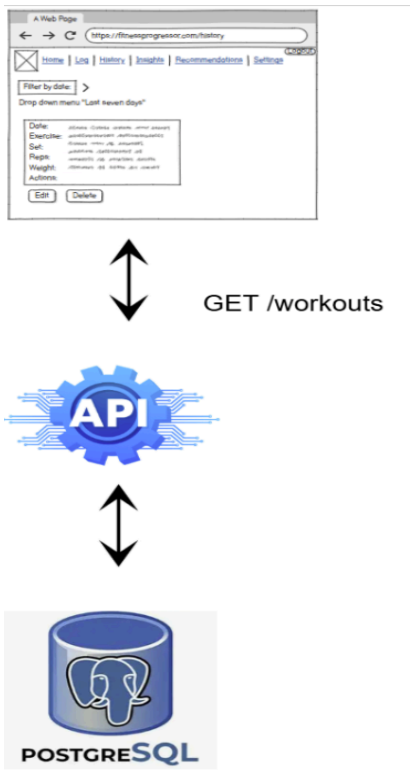
Success Response (200):

```
json
[
  {
    "workoutId": 10,
    "workoutDate": "2026-03-15",
    "sets": [
      { "exerciseName": "Bench Press", "reps": 8, "weight": 65 }
    ]
  }
]
```

Error Response (400):

```
json
{
  "error": "Invalid date range."
}
```

Diagram:



6. Update a Workout

Method: PUT
URL: /workouts/{workoutId}

Purpose:
Edit an existing workout from the History page.

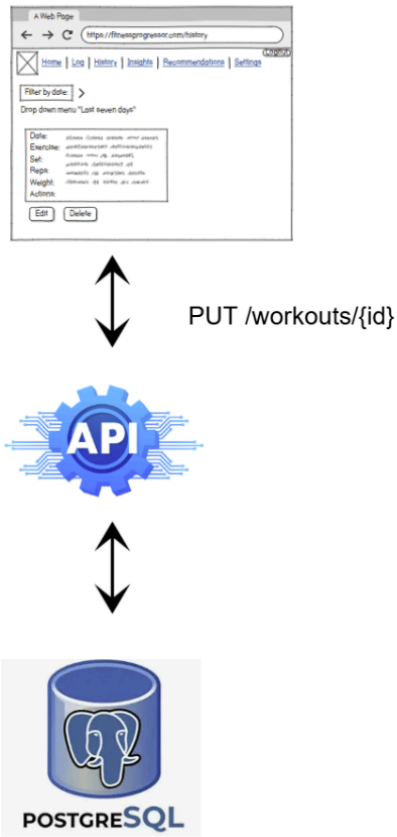
Success Response (200):

```
json
{
  "message": "Workout updated successfully."
}
```

Error Response (404):

```
json
{
  "error": "Workout not found."
}
```

Diagram:



7. Delete a Workout

Method: DELETE
URL: /workouts/{workoutId}

***Purpose*:**
Delete a workout from the History page.

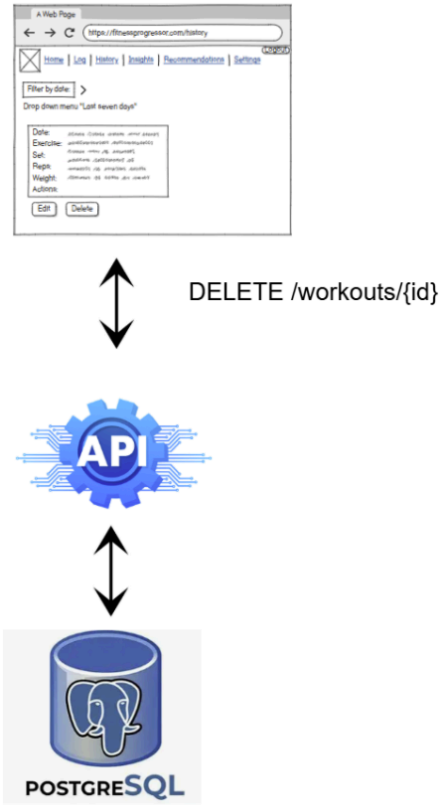
Success Response (200):

```
json
{
  "message": "Workout deleted successfully."
}
```

Error Response (403):

```
json
{
  "error": "You are not authorized to delete this workout."
}
```

Diagram:



6. Dashboard Route (Home Page)

8. Get Dashboard Summary

Method: GET
URL: /dashboard/summary

Purpose:
Provide weekly/monthly charts and summary metrics for the **Home** page.

Persona Connection:

- Mia: sees simple progress
- Daniel: sees trends
- Emily: quick weekly snapshot

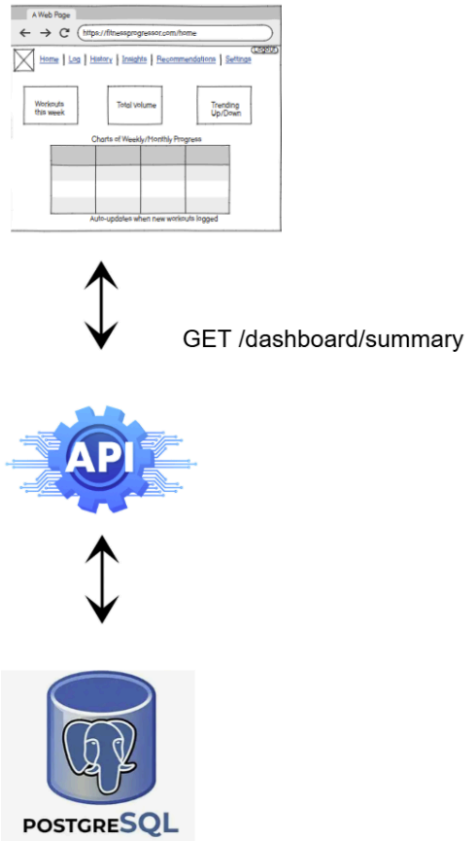
Success Response (200):

```
json
{
  "workoutsThisWeek": 3,
  "totalVolumeThisWeek": 14500,
  "trend": "up",
  "weeklyChart": [...],
  "monthlyChart": [...]
}
```

Error Response (404):

```
json
{
  "error": "No workout data found."
}
```

Diagram:



7. Insights Route (Insights Page)

9. Get Insights

Method: GET
URL: /insights

Purpose:

Serve the **Insights** page with:

- Plateau detection
- Trend charts
- Goal forecasting

Persona Connection:

- Mia: understands if she's improving
- Daniel: identifies plateaus
- Emily: quick insight without manual analysis

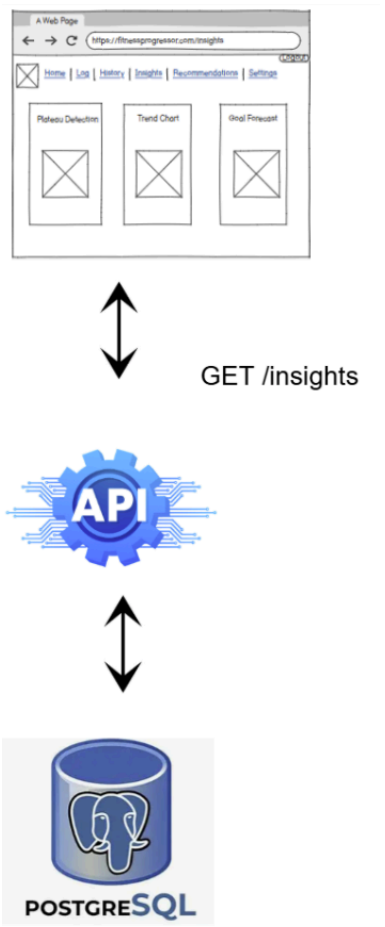
Success Response (200):

```
json
{
  "plateauDetection": [...],
  "trendChart": [...],
  "goalForecast": [...]
}
```

Error Response (400):

```
json
{
  "error": "Insufficient data to generate insights."
}
```

Diagram:



8. Recommendations Route (Recommendations Page)

10. Get Recommendations

Method: GET
URL: /recommendations

Purpose:
Serve the **Recommendations** page with personalized suggestions.

Persona Connection:

- Mia: simple actionable advice
- Daniel: performance optimization
- Emily: fast weekly guidance

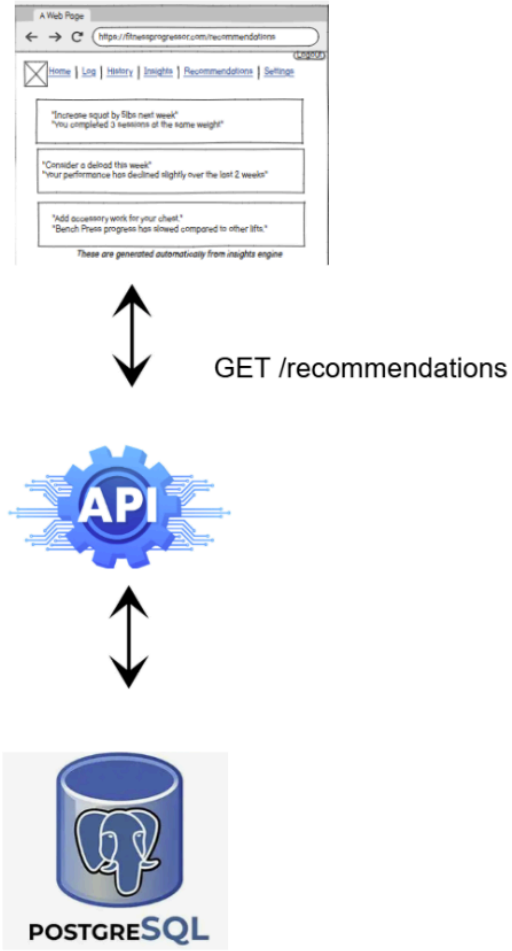
Success Response (200):

```
json
[
  {
    "exerciseName": "Squat",
    "text": "Increase squat by 5lbs next week.",
    "reason": "You completed 3 sessions at the same weight."
  }
]
```

Error Response (404):

```
json
{
  "error": "No recommendations available."
}
```

Diagram:


A Web Page
Home | Log | History | Insights | Recommendations | Settings
"Increase squat by 5lbs next week"
"You completed 3 sessions at the same weight"
"Consider a deload this week"
"Your performance has declined slightly over the last 2 weeks"
"Add accessory work for your chest."
"Bench Press progress has slowed compared to other lifts."
These are generated automatically from insights engine
GET /recommendations
API
POSTGRESQL

9. UI → Endpoint Mapping Diagram

UI → Endpoint Mapping Diagram

Home Page
→ GET /dashboard/summary

Log Page
→ POST /workouts

History Page
→ GET /workouts
→ PUT /workouts/{id}
→ DELETE /workouts/{id}

Insights Page
→ GET /insights

Recommendations Page
→ GET /recommendations

Settings Page
→ GET /users/{id}
→ PUT /users/{id}

10. Conclusion

The FitnessProgressor service layer provides a complete and RESTful interface between the frontend and the PostgreSQL database. Each endpoint corresponds directly to a user action in the MVP, ensuring that the UI, business logic, and data remain cleanly separated. This architecture supports scalability, maintainability, and the analytical features that differentiate FitnessProgressor from simple workout-logging apps.